Language as a latent port for LeWorldModel control on Push-T: a CPU probe of block angle

MMAGAI team
Skoltech

Abstract

This note asks whether block angle on Push-T is linearly readable in the frozen official LeWorldModel latent, including after a 180 degree turn. The world model is the public `quentinll/lewm-pusht` checkpoint. No language model sits inside the JEPA, and no goal image is generated from text. Words are treated as a port onto objects that already live in the 192-dimensional embedding z. On a CPU-only Xeon, a ridge probe from z to $(\cos\theta, \sin\theta)$ has held-out mean absolute circular error 12.08 degrees on scripted 180 degree turns and 14.58 degrees on frames within 30 degrees of angle 0. Under the pre-registered rule, 180 degrees is not a weak slice of this latent, so LeWorldModel is not finetuned. A least-squares displacement $f(\text{text}) \in \mathbb{R}^{192}$ fit on the same turns does not beat predicting the mean residual scale of Δz.

Keywords: LeWorldModel, JEPA, Push-T, linear probe, language interface.

1 Introduction

LeWorldModel (LeWM) is a joint-embedding predictive architecture trained from pixels with a next-embedding loss and a Gaussian regularizer [Maes et al., 2026]. The Push-T checkpoint used here was not trained with language, and this note does not put an LLM inside that JEPA. The control story we want is closer to the latent-world-model programme in LeCun [2022]: plan in a compact state, and attach language at that state rather than at pixels. A practical port has to know which physical quantities are already linear in z. Push-T's T-block angle θ is the quantity tested here, because a half-turn is a common failure mode for appearance-based embeddings that confuse θ with $\theta + \pi$. The experiment is a frozen encoder, a scripted angle set, and a linear readout. Cross-entropy method planning was not run.

2 Related Work

LeCun [2022] argues for autonomous intelligence built around a predictive world model in representation space, with separate modules for cost and

action. LeWM is one pixel-trained instance of that pattern: the published model encodes a 224×224 RGB frame to a 192-dimensional vector and predicts future embeddings from actions [Maes et al., 2026]. GRIF aligns language with the change between observations rather than with a goal image, then trains a policy on that interface [Myers et al., 2023]. The displacement head in Section 3 is the same idea restricted to a frozen LeWM latent and to four template commands. VL-JEPA predicts a continuous embedding of target text instead of autoregressive tokens [Chen et al., 2025]. We do not train that model. We only ask whether a text string can name a displacement that the frozen Push-T latent already contains.

3 Methods

3.1 Hardware and frozen encoder

The machine is an Intel Xeon E3-1270 v2 with 16 GB of RAM and no AVX2 or BMI2. The GPU is an AMD RX 580 and was not used. PyTorch is 2.14.0+cpu. `torch.cuda.is_available()` is false. stable-worldmodel is 0.1.1 and transformers is 4.57.6. Transformers 5 renames ViT parameters and cannot load this checkpoint. The call is `load_pretrained` on the repository id `quentinll/lewm-pusht`. `STABLEWM_HOME` points at the project data directory. The module is set to eval mode on CPU. Pixels follow `scripts/sanity_encode.py`: uint8 HWC, scale to $[0, 1]$, ImageNet mean and standard deviation, resize to 224, then a time axis so the tensor is $(B, 1, 3, 224, 224)$. `encode({"pixels": pixels})` returns `info["emb"]` with shape $(B, T, 192)$. One frame took 0.123 seconds, so the full set of 440 frames was encoded in batches of 8. No frame was dropped for speed. `eval.py` and CEM were not run. The 13 GB Lance dataset and the 43 GB HDF5 archive were not downloaded.

3.2 Push-T, headless

The environment id is `swm/PushT-v1` with `render_mode` set to `rgb_array`. The SDL video driver is dummy. The matplotlib backend is Agg. The pygame support prompt is hidden. The MuJoCo GL backend variable is egl. `render_mode="human"` was not used. A seed-0 reset is saved as `reset_api.json`. The unwrapped class is `PushT`. Block pose in the info dict is $(x, y, \theta_{\text{raw}})$ in pixels and radians. `obs["state"]` is length 7: agent xy, block xy, block angle wrapped to $[0, 2\pi)$, and agent velocity. The goal overlay in that dump is fixed at $(256, 256, \pi/4)$ because goal factors are not in the default randomization set. Block angle is randomized on reset. The variation-space initial value of block angle is 0, which is the origin used below for "default pose". In-place turns call `PushT._set_state` on the reset state with the block angle replaced by $\theta + \Delta\theta$ and agent velocity set to 0. The

setter writes the pymunk body and steps the space once. Measured xy after that step moved by 0.13 pixels on average and 3.78 pixels at most, against the paired random frame.

3.3 Frame set and split

Eighty episodes each contribute a random reset and three turns tagged `rot90`, `rot180`, and `rot_m90` ($\Delta\theta \in \{+\pi/2, \pi, -\pi/2\}$). One hundred and twenty further episodes are random resets only, so the official pose distribution is not only the bases of the turns. The total is 440 frames. Each row stores $z \in \mathbb{R}^{192}$, wrapped θ in radians and degrees, block xy, episode id, and the tag. Files are `data/probes/pusht_theta_probe.npz` and `data/probes/pusht_theta_probe.csv`. PNGs were not written. Episodes are split 75/25 within each episode kind, seed 0, giving 150 train episodes (330 frames) and 50 test episodes (110 frames). Frames from one episode never cross the split.

3.4 Angle probe

The probe predicts $(\cos\theta, \sin\theta)$, not the raw angle. Ridge regression with $\alpha = 1$ is the primary fit. Ordinary least squares is reported beside it. Predicted angle is $\operatorname{atan2}(\widehat{\sin\theta}, \widehat{\cos\theta})$. Circular error is the wrapped difference of that angle and the measured θ, in degrees. Mean absolute circular error (MAE) is the summary. The decision rule, fixed before looking at a preferred slice, is: if held-out MAE on `rot180` frames is more than twice the MAE on frames with wrapped $|\theta| < 30°$, or if the `rot180` MAE exceeds 30 degrees, say that 180 degrees is weakly represented and a rental finetune of LeWM is justified. Otherwise say that LeWM is not finetuned and only the language port is trained. Plots use the matplotlib Agg backend and are saved as PDF.

3.5 Language-port stub

`scripts/language_port.py` defines two interfaces that share one map f from a command string to $\mathbb{R}^{192}$. Displacement is

$$\hat{z}_g = z_t + f(\text{text}). \tag{1}$$

Energy is

$$E(z_t, z_{t+H}, \text{text}) = \|(z_{t+H} - z_t) - f(\text{text})\|^2. \tag{2}$$

The command list is "rotate T 90", "rotate T 180", "rotate T -90", and "put T on the target". `CLIPTextModel` and `CLIPTokenizer` import on this CPU. CLIP weights were not downloaded. The fitted embedder is a four-way one-hot bag of commands. $f(\text{text}) = W^\top e(\text{text})$ is least squares of $\Delta z = z_{t+H} - z_t$ on that one-hot, using train episodes only. Pairs are the three turns inside each rotation episode. A pair is kept when the measured turn is within 20

Table 1: Held-out mean absolute circular error in degrees. Tags are the collection labels. Near 0 means wrapped $|\theta| < 30°$.

Probe	random	rot90	rot180	rot−90
Ridge $\alpha = 1$	12.36	16.17	12.08	16.50
OLS	19.91	16.90	20.06	17.09

Probe	all test	near 0
Ridge $\alpha = 1$	13.75 ($n = 110$)	14.58 ($n = 19$)
OLS	18.88 ($n = 110$)	26.09 ($n = 19$)

degrees of the command and the block moves at most 5 pixels. All 240 turn pairs passed. "put T on the target" has no same-xy pair in this collection. Its column of W is the zero vector from the least-squares null space, and its residual is not measured. No text-to-image model is trained. The predictor is not conditioned on text. `AutoCostModel` is not used.

4 Results

Table 1 is the held-out circular MAE. On the ridge probe, `rot180` MAE is 12.08 degrees ($n = 20$) and the near-default slice $|\theta| < 30°$ has MAE 14.58 degrees ($n = 19$). The ratio is $12.08/14.58 \approx 0.83$. The `rot180` MAE is below 30 degrees. The rule therefore says: do not finetune LeWM; only train the language port. Ordinary least squares agrees on that sentence: its `rot180` MAE is 20.06 degrees and its near-default MAE is 26.09 degrees. OLS test MAE on all 110 held-out frames is 18.88 degrees against a train MAE of 6.59 degrees, with train $(\cos, \sin)$ $R^2 = 0.958$. Ridge test MAE is 13.75 degrees against a train MAE of 7.95 degrees, with train $R^2 = 0.935$. Ridge is the probe cited by the decision because the test error is lower. Figure 1 shows held-out predicted angle against wrapped true angle. Figure 2 shows the signed circular error inside eight bins of true θ. Figure 3 and Table 2 show MAE against absolute angle from 0. No bin mean exceeds 30 degrees under ridge. The noisiest ridge bin in that table is $[15, 30)$ degrees, at 18.58 degrees with $n = 6$.

The language port is a separate measurement. Train RMSE of $\Delta z - f(\text{text})$ is 1.089 per latent coordinate. The RMS of Δz itself is 1.099, so the ratio is 0.991. Mean ℓ_2 residual versus mean ℓ_2 of Δz is 13.41 versus 13.51 for "rotate T 90", 16.75 versus 16.89 for "rotate T 180", and 13.91 versus 13.99 for "rotate T -90", each on 60 train pairs. Held-out RMSE is 1.197 on 60 pairs. Mean held-out energy is 274.97. "put T on the target" is not measured, and $\|f\|$ for that string is 0. A single vector per command does not account for the turn. CEM success rate is not measured.

Table 2: Held-out ridge MAE by absolute wrapped angle from the default angle 0.

$\lvert\theta\rvert$ bin (deg)	n	MAE (deg)
$[0, 15)$	13	12.74
$[15, 30)$	6	18.58
$[30, 45)$	11	12.51
$[45, 60)$	9	9.96
$[60, 90)$	16	13.42
$[90, 135)$	25	14.51
$[135, 180]$	30	14.36

5 Discussion

The angle readout does not get worse at a half turn. Ridge MAE at `rot180` is slightly smaller than MAE near angle 0, and both sit near 12–15 degrees. That is the region the decision rule treats as comparable, so the checkpoint's encoder is left frozen. The same latent does not give a position-independent displacement for the template commands: $f(\text{text})$ removes about one percent of the RMS of Δz. Angle can be read by a linear map that sees z, while the difference $z(\theta + \Delta\theta) - z(\theta)$ still depends on where the block sits and on the base angle. A language port that is only a lookup of Δz is not a controller. Training that port, if it is trained further, has to be a richer map than $f(\text{text})$ alone. This run does not justify spending a rental on a LeWM finetune for θ. The rental that is justified is a 24 GB NVIDIA host for CEM and for the Push-T training set, with `transformers` kept below version 5 until the checkpoint is re-exported. The Lance and `kornia_rs` wheels still SIGILL on this Xeon. They were not required for the probe. CLIP class imports succeeded and CLIP weights were not fetched. Replacing the one-hot embedder with a frozen CLIP text tower is future work on a machine where that download is wanted.

6 Conclusion

On 440 headless Push-T frames, a frozen official LeWM encodes each frame to $z \in \mathbb{R}^{192}$ and a ridge probe reads θ with held-out MAE 13.75 degrees. Scripted 180 degree frames are not the weak bin. LeWM is not finetuned on the evidence of this probe. The least-squares language displacement is fit and reported, and it does not explain Δz. Full planning and the large datasets were left for a later GPU machine.

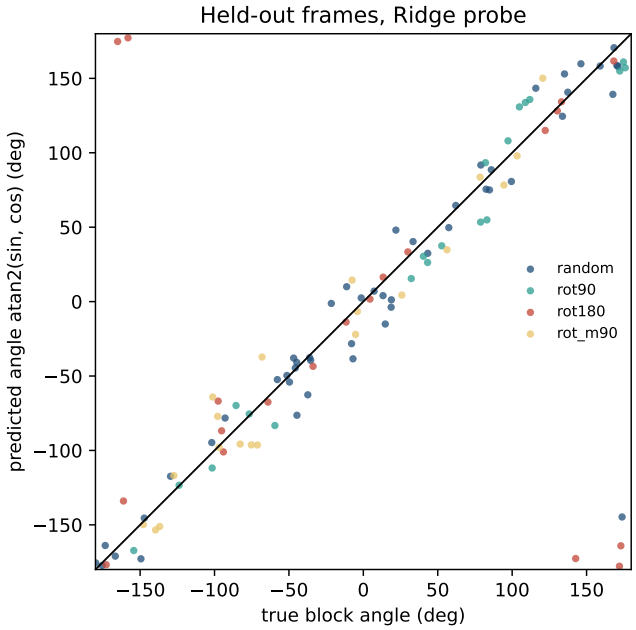


Figure 1: Held-out ridge predictions. Both axes are wrapped to $[-180, 180]$ degrees. The line is equality.

A Reset API

Seed 0 of `swm/PushT-v1` returns an `OrderEnforcing` wrapper around `PushT`. Observation keys are `proprio` and `state`. Info keys include the block pose, the goal pose, the goal state, agent position and velocity, a contact count, and a goal image. `goal` is a $224 \times 224 \times 3$ uint8 image and was not stored in the probe table. The goal pose in that dump is $(256, 256, 0.785)$. Block pose in that dump is about $(383.77, 318.04, 4.97)$. Turns use `_set_state`, which is present on the unwrapped env.

References

Delong Chen, Mustafa Shukor, Théo Moutakanni, Willy Chung, Jade Yu, Tejaswi Kasarla, Allen Bolourchi, Yann LeCun, and Pascale Fung. VL-JEPA: Joint embedding predictive architecture for vision-language. *arXiv preprint arXiv:2512.10942*, 2025. URL https://arxiv.org/abs/2512.10942.

Yann LeCun. A path towards autonomous machine intelligence. OpenReview, 2022. URL https://openreview.net/forum?id=BZ5a1r-kVsf. Version 0.9.2, 27 June 2022.

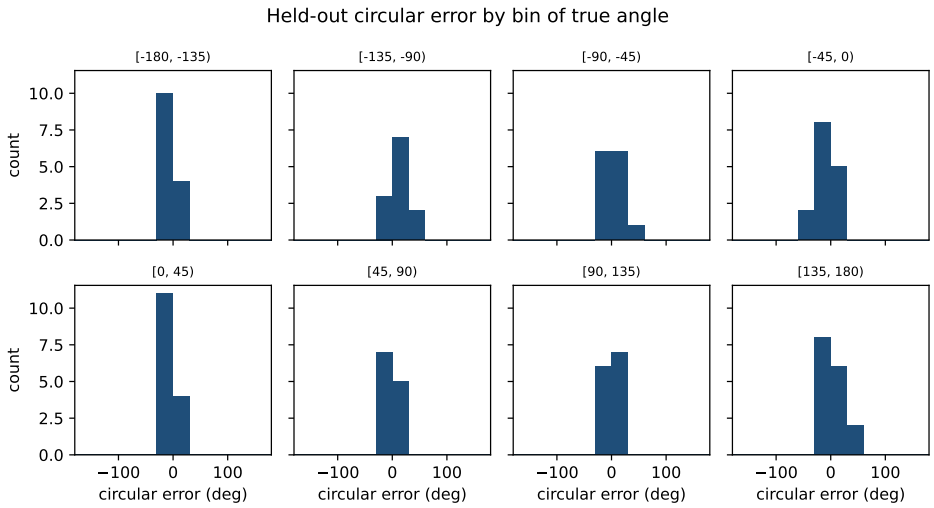


Figure 2: Held-out signed circular error, one panel per bin of true wrapped angle.

Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. URL https://arxiv.org/abs/2603.19312.

Vivek Myers, Andre He, Kuan Fang, Homer Walke, Philippe Hansen-Estruch, Ching-An Cheng, Mihai Jalobeanu, Andrey Kolobov, Anca Dragan, and Sergey Levine. Goal representations for instruction following: A semi-supervised language interface to control. In *Conference on Robot Learning*, 2023. URL https://arxiv.org/abs/2307.00117. arXiv:2307.00117.

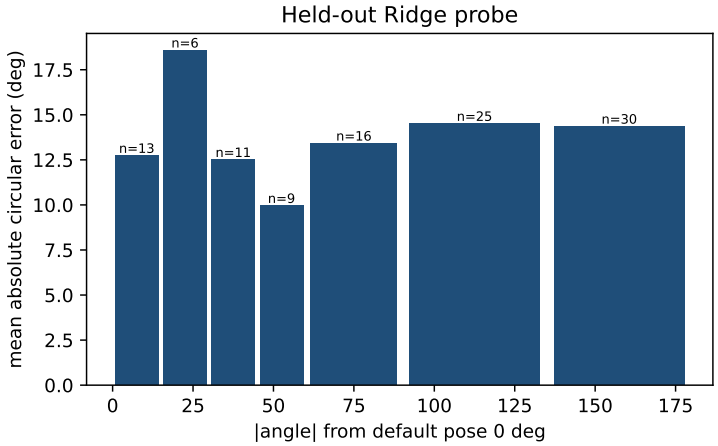


Figure 3: Held-out ridge MAE against $|\theta|$ from angle 0. Bar labels are bin counts.